

O problema da coloração de grafos com Algoritmo Backtracking e Guloso

Complexidade de Algoritmos

Alunos: Mariana dos Santos e Herton Silveira

1. Grafo utilizado

O exemplo utilizado disponibilizado pelo professor é uma matriz de adjacência. Dado um grafo G com n vértices, podemos representá-lo em uma matriz $n \times n$ $A(G)=[a_{ij}]$. O valor a_{ij} guarda informações sobre como os vértices v_i e v_j estão relacionados (isto é, informações sobre a adjacência de v_i e v_j). Caso $a_{ij} = 1$, então v_i e v_j estão conectados por uma aresta.

2. Implementação dos algoritmos

As versões dos algoritmos usadas foram:

2.1 Backtracking

```
// Função recursiva para tentar colorir o grafo com 'm' cores
bool colorir_grafo_bt(int grafo[V][V], int cores[], int vertice, int m) {
    if (vertice == V) {
        return true; // Todos os vértices foram coloridos
    }

    // Tenta colorir o vértice com diferentes cores (de 1 a m)
    for (int cor = 1; cor <= m; cor++) {
        if (pode_colorir_bt(grafo, cores, vertice, cor)) {
            cores[vertice] = cor;
            if (colorir_grafo_bt(grafo, cores, vertice + 1, m)) {
                return true;
            }
            cores[vertice] = 0;
        }
    }
    return false; // Não é possível colorir com m cores
}
```

Abaixo, o algoritmo usado para rodar o Backtracking com valor cada vez mais próximo do número mínimo de cores possíveis:

```

while (menor_m_bt > 1) {
    // Inicializa as cores como 0 para cada nova tentativa de 'm'
    for (int i = 0; i < V; i++) {
        cores_bt[i] = 0;
    }

    printf("Tentando colorir com %i cores\n", menor_m_bt);
    if (colorir_grafo_bt(grafo, cores_bt, 0, menor_m_bt)) {
        printf("Foi possível colorir!\n");
        printf("Coloração do grafo (Backtracking): \n");
        for (int i = 0; i < V; i++) {
            printf("Vértice %d -> Cor %d\n", i, cores_bt[i]);
        }
    }
    menor_m_bt--;
}

```

2.1 Guloso

```

void colorir_grafo_guloso(int grafo[V][V], int cores[]) {
    // Inicializa todas as cores como 0 (sem cor)
    for (int i = 0; i < V; i++) {
        cores[i] = 0;
    }

    int max_cores_usadas = 0; // Para rastrear o número máximo de cores necessárias

    // Itera sobre cada vértice
    for (int vertice = 0; vertice < V; vertice++) {
        // Encontra a menor cor disponível para o vértice atual
        int cor_escolhida = encontrar_menor_cor_disponivel(grafo, cores, vertice, max_cores_usadas);
        cores[vertice] = cor_escolhida;

        if (cor_escolhida > max_cores_usadas) {
            max_cores_usadas = cor_escolhida;
        }
    }
}

```

3. Resultados encontrados

O algoritmo de Backtracking tenta colorir com V cores, onde V é o tamanho do grafo, depois com $V - 1$, com $V - 2$ e assim por diante até testar uma quantidade de cores que aumenta drasticamente o tempo de computação. O problema de coloração de grafos para encontrar o número cromático é um problema NP-completo. Isso significa que, até onde sabemos, não existe um algoritmo que o resolva em tempo polinomial. Algoritmos exatos como o backtracking têm uma complexidade de tempo que cresce exponencialmente com o tamanho do grafo no pior caso.

Abaixo os resultados encontrados:

Algoritmo	Cores Utilizadas	Observações
Guloso	10	Rápido, mas não garante o mínimo de cores.
Backtracking	8	Mais lento, mas encontrou uma coloração com 8 cores. Este é o número mínimo encontrado até agora pelo backtracking.